

## **Week 7: Language Comparison**

Aaron Foster

Southern New Hampshire University

CS 660: Advanced Programming Language Theory

Prof. Shadha Tabatabai

Aug 23rd, 2026

**Domain Significance: High-Volume Data Analytics**

High-volume data analytics is the practice of ingesting, transforming, and querying continuous streams of event data at a scale and speed that makes the result useful while it is still current. It is the engine behind fraud detection, recommendation systems, observability platforms, logistics routing, and real-time personalization. The domain matters because its economics are unusual: the cost of the system scales with the volume of data rather than with the number of users, and the value of an answer decays rapidly with latency. A fraud score delivered two seconds late is not a slightly worse product; it is frequently a useless one, because the transaction has already cleared.

Those two properties make language selection unusually consequential here. In a domain where per-record processing cost is multiplied by billions of records, a constant-factor difference in execution speed translates directly into server count, energy consumption, and operating expense. Pereira et al. (2021) quantified this across 27 languages using the Computer Language Benchmark Game, and found the spread between the extremes to be enormous rather than marginal: normalized against the most efficient language, Python consumed 75.88 times more energy and 71.90 times more time, while Go consumed 3.23 times more energy and 2.83 times more time. Expressed as a direct ratio, Python required roughly 23 times the energy and 25 times the execution time of Go for equivalent work. In a domain that runs continuously at scale, that is an infrastructure budget line, not a benchmark curiosity.

## Relevance of the Two Languages

Python and Go are both central to this domain, and they arrived there from opposite directions. Python became the default language of data work through its ecosystem: NumPy, pandas, Apache Arrow, PyTorch, and the broader scientific stack made it the language in which analytical logic is expressed, and the 2025 Stack Overflow Developer Survey found Python to be the fastest-growing major language, rising seven percentage points year over year across more than 49,000 respondents (Stack Overflow, 2025). Any analytics organization already has Python in it.

Go entered the domain from the infrastructure side. It was designed at Google explicitly for networked services running on multicore hardware, and it now underpins much of the machinery analytics platforms are built on, including Docker, Kubernetes, Prometheus, and InfluxDB.

Its relevance is concrete rather than theoretical: Uber built its highest queries-per-second service in Go, sustaining a peak of 170,000 queries per second on 40 machines at 35% CPU utilization with 95th-percentile latency under five milliseconds, and chose Go over an interpreted alternative specifically because the workload was CPU-intensive and needed genuine parallelism (Wei, 2016). Notably, Wei observed that Uber had historically been "mostly a Node.js and Python shop" — the migration pressure this paper examines is the same one that the firm encountered.

## Method

The comparison that follows draws on published sources and on original measurements taken for this assignment. Because secondary benchmarks vary in hardware, compiler version, and implementation quality, a controlled experiment was run to corroborate them.

Three sorting algorithms — quick sort, merge sort, and heap sort — were implemented in both languages as line-for-line ports of one another: the same median-of-three pivot, the same Hoare partition, the same bottom-up heap construction. Both programs read the same 100,000 integers from a shared file rather than generating their own, since the two languages have different pseudorandom generators and "same seed" would not have meant "same data."

Both harnesses made a fresh copy of the input before each timed iteration outside the timed region, forced garbage collection before each iteration, ran one untimed warm-up, and validated every result against a known-sorted copy. All measurements were taken on one machine: an Apple M4 Max with 14 cores and 36 GB of memory, running CPython 3.13.0 and Go 1.27.0. Holding the algorithm, the data, the methodology, and the hardware constant leaves the language and its runtime as the only variable.

Comparison of Python and Go

Table 1

| Criterion        | Python (CPython 3.13)                                                                                                                                                                  | Go (1.27)                                                                                                                                                                   |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Runtime behavior | <p>Interpreted bytecode on a stack VM;</p> <p>dynamic typing resolved at runtime;</p> <p>reference counting plus cycle-detecting GC; GIL serializes bytecode execution to one core</p> | <p>Compiled to native machine code ahead of time; static typing resolved at compile time; concurrent tri-color mark-and-sweep GC; goroutines scheduled across all cores</p> |
| Performance      | <p>83–139 ms for the three sorts (measured);</p> <p>competitive only when work is delegated to C libraries — list.sort at 7.97 ms</p>                                                  | <p>5.4–6.8 ms for the identical algorithms (measured), a 14–26x advantage; 3.07 ms with goroutine parallelism</p>                                                           |

|                                  |                                                                                                                                                          |                                                                                                                                                                          |
|----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Tooling ecosystem</b>         | <b>Unmatched analytical depth — NumPy, pandas, Arrow, scikit-learn, PyTorch; fragmented packaging (pip, conda, poetry, uv); optional typing via mypy</b> | <b>Smaller analytical library set; exceptional built-in tooling — go build, test, fmt, vet, race detector, pprof — unified and versioned with the language</b>           |
| <b>Ease of deployment</b>        | <b>Requires matching interpreter and all dependencies on the target host; 275 MB install measured; 11.9 ms cold start, 44.7 ms with NumPy</b>            | <b>Single statically linked binary, 3.7 MB measured, no runtime on target; 2.2 ms cold start; trivial cross-compilation</b>                                              |
| <b>Long-term maintainability</b> | <b>Rapid to write; correctness of large refactors depends on test coverage and</b>                                                                       | <b>Compiler catches type and interface errors before deployment; enforced formatting; strong backward-compatibility guarantee; smaller talent pool but short ramp-up</b> |

|  |                                                        |  |
|--|--------------------------------------------------------|--|
|  | <b>optional type hints;<br/>very large talent pool</b> |  |
|--|--------------------------------------------------------|--|

*Note. Performance and deployment figures in this table are original measurements taken for this assignment on a single Apple M4 Max running CPython 3.13.0 and Go 1.27.0. Method described above; full results in Table 2 and the Appendix.*

## Runtime Behavior

The two runtimes differ in when work happens. CPython compiles source to bytecode and executes it in a dispatch loop, so every comparison, index, and loop increment costs one or more bytecode dispatches plus dynamic type resolution. Go compiles to native machine code ahead of time, so the same comparison becomes a machine instruction on values already in registers. This is the origin of the performance gap discussed below, and it is structural rather than incidental.

The more consequential difference for analytics is concurrency. CPython's global interpreter lock permits only one thread to execute Python bytecode at a time, so a pure-Python workload cannot use more than one core regardless of how many are available.

Go's runtime multiplexes goroutines onto GOMAXPROCS operating-system threads and executes them genuinely in parallel. The measurement makes the difference concrete: adding goroutine parallelism to the Go merge sort reduced it from 6.82 ms to 3.07 ms, a 2.2x improvement on the same 14-core machine, while the structurally identical change is unavailable to the Python implementation.

This limitation is being addressed, and an honest comparison must say so. PEP 703 was accepted and a free-threaded build without the GIL shipped beginning with CPython 3.13 (Gross, 2023). The single-threaded cost of that build has fallen substantially; current Python documentation reports average overhead "from about 1% on macOS aarch64 to 8% on x86-64 Linux systems" (Python Software Foundation, 2026).

The GIL is therefore a diminishing constraint rather than a permanent one. What free threading does not change is the per-operation interpreter cost: it removes the ceiling on parallelism without making any individual Python operation faster. A workload that is 14 times slower per core remains 14 times slower per core once it is allowed to use several. Memory management also diverges in a way that matters for latency.

Go's collector runs concurrently with the application and is tuned specifically to minimize pause times, though the Go Authors (2022) note it still consumes roughly 25% of CPU capacity during its mark phase and introduces brief stop-the-world transitions. CPython's reference counting reclaims most objects deterministically the moment they become unreachable, which yields predictable behavior, but each reference count update is itself work performed on every object operation.

## **Performance**

The controlled experiment produced the results in Table 2. The algorithms, the input, the methodology, and the hardware were identical across both columns.

Table 2

| <b>Workload<br/>(100,000<br/>identical<br/>integers)</b>          | <b>Python</b>    | <b>Go</b>      | <b>Go advantage</b> |
|-------------------------------------------------------------------|------------------|----------------|---------------------|
| <b>Quick sort<br/>(hand-written,<br/>identical<br/>algorithm)</b> | <b>83.05 ms</b>  | <b>5.78 ms</b> | <b>14.4x</b>        |
| <b>Merge sort<br/>(hand-written,<br/>identical<br/>algorithm)</b> | <b>113.92 ms</b> | <b>6.82 ms</b> | <b>16.7x</b>        |
| <b>Heap sort<br/>(hand-written,<br/>identical<br/>algorithm)</b>  | <b>139.14 ms</b> | <b>5.40 ms</b> | <b>25.8x</b>        |
| <b>Standard-library sort</b>                                      | <b>7.97 ms</b>   | <b>7.09 ms</b> | <b>1.12x</b>        |

|                                                 |                            |                |             |
|-------------------------------------------------|----------------------------|----------------|-------------|
| <b>Merge sort with thread-level parallelism</b> | <b>not available (GIL)</b> | <b>3.07 ms</b> | —           |
| <b>Memory for the input collection</b>          | <b>3.45 MB</b>             | <b>0.76 MB</b> | <b>4.5x</b> |

*Note. Original measurements; mean of five timed iterations after one untimed warm-up, Apple M4 Max, CPython 3.13.0, Go 1.27.0. Python figures use tracemalloc for memory; Go figures use runtime.MemStats.*

The fourth row is the most important line in this paper, and it inverts the conclusion the first three rows appear to support. When each language sorts using its own standard library, the difference collapses from roughly twenty-fold to twelve percent — 7.97 ms for Python against 7.09 ms for Go. The reason is that CPython's `list.sort` is not written in Python. It is Timsort implemented in C, and when Python calls it, essentially no Python executes.

This reframes the comparison entirely. The question is not whether Go is faster than Python in the abstract; it is what proportion of a given workload is executed by the interpreter rather than delegated to compiled code. Where analytical work can be expressed as vectorized calls into NumPy, Arrow, or DuckDB, Python's effective performance approaches Go's, because the numerical kernel is compiled C in both cases. Where the work is custom element-wise logic that resists vectorization — bespoke parsing, irregular business rules, per-record branching — the interpreter executes it directly and the 14- to 26-fold penalty measured above applies in full.

These original results also corroborate the published literature closely. Pereira et al. (2021) separately benchmarked merge sort and quick sort in both languages using Rosetta Code implementations, reporting merge sort at 6 ms in Go against 86 ms in Python, and quick sort at 9 ms against 61 ms. Those ratios of roughly 14x and 7x bracket the 14.4x and 16.7x measured here on entirely different hardware, which suggests the gap is a property of the runtimes rather than of any particular machine.

### **Tooling Ecosystem**

This criterion favors Python decisively, and no measurement taken for this paper changes that. The analytical ecosystem Python has accumulated over two decades — NumPy, pandas, Arrow, Polars, scikit-learn, PyTorch, Dask — has no equivalent in Go. For statistical modeling, machine learning, and exploratory analysis, choosing Go means either writing primitives that already exist in Python or calling out to another runtime. Python's adoption trajectory reinforces this: it was the fastest-growing major language in the 2025 Stack Overflow Developer Survey, rising seven percentage points against Go's two (Stack Overflow, 2025).

Go's advantage is in a different category of tooling, and it is easy to undervalue when comparing library counts. Go ships a unified, versioned toolchain with the language: `go build`, `go test`, `go fmt`, `go vet`, a race detector, and the `pprof` profiler are standard rather than chosen. Python's equivalents are a matter of selection and negotiation — `pip`, `conda`, `poetry`, or `uv` for packaging; `pytest` or `unittest` for testing; `black`, `ruff`, or `flake8` for style; `mypy` for optional typing. Each is capable, but the team must assemble and maintain the combination, and that overhead recurs on every new service and every new hire. Go's race detector deserves specific mention in

a domain built on concurrency: it catches data races at test time, a class of defect that is notoriously difficult to reproduce in production.

### **Ease of Deployment**

Go compiles to a single statically linked binary with no runtime dependency on the target host. The binary built for this assignment measured 3.7 MB and linked only two system libraries. Deploying it requires copying one file, and cross-compiling for a different operating system or architecture is a matter of setting two environment variables.

Python requires a matching interpreter and the full dependency tree present wherever the code runs. The CPython installation measured for this comparison occupied 275 MB, and a single common dependency, NumPy, added 29 MB. Cold-start times followed the same pattern: the Go binary launched in a median of 2.2 ms, bare CPython in 11.9 ms, and CPython importing NumPy in 44.7 ms — 5.4 and 20.4 times Go's figure respectively. For a long-running service started once, this is immaterial. For containerized workers that scale elastically with load, or for serverless functions billed per invocation, a 74-fold difference in image size and a 20-fold difference in cold start are direct operating costs.

### **Long-Term Maintainability**

Static typing is the central distinction. Go's compiler rejects type and interface errors before code is deployed, which converts a category of production failure into a build failure. In Python those errors surface at runtime unless the team has invested in type hints and a checker such as mypy, and that investment is optional, partial, and easy to let lapse.

In a data pipeline where a malformed record may reach a rarely-exercised branch hours into a batch, the difference between a compile-time and a runtime failure is substantial.

Go further constrains variation deliberately. Enforced formatting through `gofmt` eliminates style disagreement, and the language's small feature set means most Go code looks alike regardless of who wrote it — an asset when a service outlives its original author.

Go's compatibility promise means code written years ago generally still builds. Python's greater expressiveness accelerates initial development but widens the range of idioms a maintainer must recognize, and its ecosystem moves faster, which raises the ongoing cost of dependency maintenance. The counterweight is talent: Python's far larger developer population makes hiring and replacing engineers easier, which is itself a maintainability property.

## **Recommendation**

For the service tier of a high-volume analytics platform — the components that ingest, validate, transform, route, and serve event data under latency and throughput commitments — I recommend Go.

The recommendation follows from the comparison rather than from general preference, and rests on three findings. First, the workload in this tier is precisely the kind Python is worst at. It is custom per-record logic — parsing, validation, routing, enrichment — that does not vectorize into library calls, so it executes in the interpreter and incurs the full 14- to 26-fold penalty measured in Table 2. The narrowing seen in the standard-library row does not apply here, because there is no compiled kernel to delegate to.

Second, this tier is where concurrency is the workload. Ingestion and serving are concurrent by nature, and Go's goroutines exploit multiple cores while the GIL prevents pure-Python code from doing so — a difference worth a measured 2.2x on a single sorting operation, and considerably more on a service handling many simultaneous requests. Uber's geofencing service demonstrates the ceiling this makes reachable: 170,000 queries per second on 40 machines at 35% CPU (Wei, 2016).

Third, the operational economics compound. Go was one of only three languages Pereira et al. (2021) placed in the Pareto-optimal set across energy, time, and memory simultaneously, alongside C and Pascal, and it ranked second of 27 languages in memory efficiency. Combined with a 3.7 MB deployment artifact against 275 MB and a 2.2 ms cold start against 44.7 ms, this compounds across every replica of every service, continuously.

A supporting consideration is that the recommendation is not all-or-nothing. The prevailing architecture in this domain is a polyglot one in which a compiled service tier feeds a Python analytical tier, and the two communicate over a columnar interchange format such as Apache Arrow or Parquet. Recommending Go for the service tier does not displace Python from the analytical work where its ecosystem is irreplaceable; it assigns each language the work its runtime is suited to.

## **Tradeoffs and Limitations**

The most significant cost of choosing Go is the analytical ecosystem surrendered. There is no Go equivalent of pandas, scikit-learn, or PyTorch with comparable maturity. If the roadmap includes statistical modeling or machine learning, that work will happen in Python regardless,

and the organization will maintain two languages, two toolchains, two deployment paths, and an interchange boundary between them. That boundary is a real and recurring cost, not a one-time integration.

Development speed is a second cost. Go's explicit error handling and static typing make it more verbose, and for exploratory work — where the requirement is understood by writing code against real data — Python's interactivity is a genuine advantage that no benchmark captures. Hiring is a third: the Stack Overflow (2025) data showing Python growing at seven points against Go's two describes a substantially larger and faster-growing talent pool.

A limitation of my own evidence should also be stated. The measurements in Table 2 use sorting, a CPU-bound, single-threaded, in-memory operation. It is a reasonable proxy for custom per-record logic, but real analytics services also spend time on network and disk I/O, where both runtimes wait on the operating system and the language gap narrows sharply.

The 14- to 26-fold advantage should be read as an upper bound on the compute portion of a workload, not as a prediction of end-to-end service latency. A further caveat is that one 100,000-element input on one machine cannot characterize either language completely; the corroboration from Pereira et al. (2021) on different hardware is what gives these figures weight beyond a single trial.

### **Factors That Would Change the Decision**

- If the workload is predominantly vectorizable. Where the pipeline is columnar aggregation expressible in NumPy, Polars, or DuckDB, Python's performance approaches

Go's — the 1.12x standard-library row rather than the 14x hand-written rows — and Python's ecosystem then wins outright.

- If machine learning is central rather than adjacent. Model training and inference are Python's domain, and splitting the stack to gain speed in the ingestion tier may cost more in integration complexity than it saves.
- If the team is already fluent in Python and small. For a team of a few engineers under delivery pressure, the productivity of a familiar language can outweigh a constant-factor runtime advantage, particularly before product-market fit.
- If free-threaded Python matures further. With single-threaded overhead already down to 1–8% (Python Software Foundation, 2026), removing the parallelism ceiling weakens one of Go's three principal advantages, though not the per-operation interpreter cost.
- If throughput requirements are modest. Below the volumes where constant factors dominate infrastructure cost, this analysis is optimizing something that is not the constraint.

## **How Language Selection Shapes Long-Term Outcomes**

Language selection is difficult to reverse. It propagates into hiring, libraries, deployment tooling, monitoring, and the accumulated knowledge of the team, so its consequences are felt years after the decision and across dimensions that are invisible at the moment it is made.

### **Scalability**

A language's constant factor determines where horizontal scaling begins. A runtime 14 times slower per core requires proportionally more machines for the same throughput, and

because the GIL confines pure-Python work to one core per process, Python scales by adding processes — each carrying a full interpreter and its own memory overhead. The scaling curve is set at the moment the language is chosen, and re-architecting to change it is far costlier than choosing differently at the start.

### **Maintainability**

Over a multi-year horizon, the dominant cost is comprehension: how quickly an engineer who did not write a service can change it safely. Static typing, enforced formatting, and a small feature set favor Go here; Python's expressiveness and optional typing shift that burden onto test coverage and documentation discipline that must be sustained indefinitely.

### **Resource Constraints**

Efficiency compounds silently. Pereira et al. (2021) measured Python at roughly 23 times Go's energy consumption, and the deployment figures measured here — 275 MB against 3.7 MB, 44.7 ms against 2.2 ms at start — multiply across every replica and every restart. In cloud environments billed by CPU-second, memory, and invocation, the language choice is a recurring line item on the infrastructure bill.

### **Community Support**

Community determines how much a team must build itself. Python's is larger and growing faster (Stack Overflow, 2025), which means most analytical problems have an existing library and most errors have an existing answer. Go's community is smaller but unusually well aligned with infrastructure work, and its standard library covers far more of that domain without

third-party dependencies — which is itself a long-term advantage, since every dependency is a maintenance obligation.

### **Team Adoption**

Adoption cost is where this decision most often fails in practice. Go is deliberately small and most engineers become productive in it quickly, but idiomatic concurrency and error handling take longer, and a team writing Python-shaped Go will realize few of the benefits above. Introducing a second language also fragments code review and on-call expertise. The realistic path is incremental: rewrite one high-volume service in Go, measure the result against the existing implementation, and expand only if the measured benefit justifies the organizational cost.

### **Conclusion**

Comparing Python and Go across runtime behavior, performance, tooling, deployment, and maintainability produces a result more specific than "Go is faster." The controlled measurement taken for this paper found Go 14 to 26 times faster when both languages executed the same hand-written algorithm, but only 1.12 times faster when each used its own standard library. That contrast identifies the real decision variable: how much of the workload the Python interpreter executes itself, as opposed to delegating to compiled code.

For the service tier of a high-volume analytics platform, that proportion is high, the work is concurrent, and the efficiency difference compounds across every replica — so Go is the correct recommendation. For the analytical tier, the proportion is low because the numerical kernels are already compiled, and Python's ecosystem is decisive. The mature answer for this

domain is therefore not a single language but a deliberate boundary between two, drawn where the interpreter stops being an acceptable cost. The value of measuring rather than assuming is that it shows exactly where that boundary belongs.

## References

Go Authors. (2022). A guide to the Go garbage collector. The Go Programming Language. <https://go.dev/doc/gc-guide>

Gross, S. (2023). PEP 703 – Making the global interpreter lock optional in CPython. Python Enhancement Proposals. <https://peps.python.org/pep-0703/>

Pereira, R., Couto, M., Ribeiro, F., Rua, R., Cunha, J., Fernandes, J. P., & Saraiva, J. (2021). Ranking programming languages by energy efficiency. Science of Computer Programming, 205, 102609. <https://doi.org/10.1016/j.scico.2021.102609>

Python Software Foundation. (2026). Python support for free threading. Python documentation. <https://docs.python.org/3/howto/free-threading-python.html>

Stack Overflow. (2025). 2025 developer survey: Technology. <https://survey.stackoverflow.co/2025/technology>

Wei, K. (2016, February 24). How we built Uber engineering's highest query per second service using Go. Uber Blog.

<https://www.uber.com/blog/go-geofence-highest-query-per-second-service/>

## **Appendix: Measurement Detail and Reproduction**

All original figures in this paper were produced by code written for it and executed on the machine described in the Method section. The measurement environment and artifacts were as follows.

| <b>Item</b>     | <b>Detail</b>                                                                                                                          |
|-----------------|----------------------------------------------------------------------------------------------------------------------------------------|
| <b>Hardware</b> | <b>Apple M4 Max, 14 cores (10 performance / 4 efficiency), 36 GB RAM</b>                                                               |
| <b>Python</b>   | <b>CPython 3.13.0, GIL enabled, macOS 26.5.1 arm64</b>                                                                                 |
| <b>Go</b>       | <b>go1.27.0 darwin/arm64, GOMAXPROCS = 14</b>                                                                                          |
| <b>Input</b>    | <b>100,000 integers in <math>[0, 10^9]</math>, generated once and read by both programs</b>                                            |
| <b>Protocol</b> | <b>5 timed iterations after 1 untimed warm-up; fresh input copy and forced GC before each iteration, both outside the timed region</b> |

|                      |                                                                                     |
|----------------------|-------------------------------------------------------------------------------------|
| <b>Validation</b>    | <b>Every result compared against a known-sorted copy before timing was reported</b> |
| <b>Python memory</b> | <b>tracemalloc peak, plus measured list footprint</b>                               |
| <b>Go memory</b>     | <b>runtime.MemStats TotalAlloc delta across one call</b>                            |

A measurement worth recording separately, because it complicates the expectation that the compiled language always uses less memory: Go's merge sort allocated 14.17 MB against Python's 1.65 MB. Go copies 8-byte int64 values when it slices, whereas Python's list slices copy 8-byte pointers to integer objects that are shared rather than duplicated.

Python's boxed object model costs memory to hold the data — 3.45 MB against 0.76 MB for the input — but refunds part of that cost during copying. The direction of the memory advantage therefore depends on whether the algorithm allocates.

**Reproduction:** `python3 gen_data.py` writes the shared input; `python3 bench_python.py` runs the Python side; `cd gobench && go run .` runs the Go side; `./deploy_metrics.sh` reproduces the artifact-size and cold-start figures.

**Acknowledgment of Generative AI Use**

Per the course AI usage guidelines: Anthropic's Claude (Opus 5) was used to help edit this report (Anthropic, 2026). The author reviewed and approved the analysis and final recommendation.

*Anthropic. (2026). Claude (Opus 5) [Large language model]. <https://claude.ai>*